

[ADVANCED] PYGAME 100 LESSONS - PART 2

Professional Game Development & Design

Master Advanced Movement, Graphics, AI, and Game Design

Build Games Inspired by DOOM, Duke Nukem, Rollercoaster Tycoon, and Command and Conquer

Created: August 27, 2026

Level: Intermediate / Advanced

Prerequisites: Complete Part 1 (100 Lessons)

Duration: ~60-80 hours of learning

TABLE OF CONTENTS

PART 1: ADVANCED MOVEMENT (Lessons 1-10)

Smooth animations, acceleration, physics

PART 2: 2.5D & PSEUDO-3D (Lessons 11-20)

DOOM-style engine techniques

PART 3: ISOMETRIC GRAPHICS (Lessons 21-30)

Rollercoaster Tycoon perspective

PART 4: ADVANCED GRAPHICS (Lessons 31-40)

Sprite sheets, particles, shading

PART 5: AI SYSTEMS (Lessons 41-50)

NPC behavior, pathfinding, decision trees

PART 6: CAMERA SYSTEMS (Lessons 51-60)

Smooth scrolling, zoom, pan, follow

PART 7: LEVEL DESIGN (Lessons 61-70)

Map generation, tile systems, design patterns

PART 8: STRATEGY MECHANICS (Lessons 71-80)

RTS elements, resource management, UI

PART 9: ADVANCED PHYSICS (Lessons 81-90)

Gravity, momentum, advanced collisions

PART 10: PROFESSIONAL ARCHITECTURE (Lessons 91-100)

State machines, event systems, optimization

LESSON 1: Acceleration vs Speed

Speed is constant movement. Acceleration makes objects gradually go faster. Use velocity variables to track momentum.

LESSON 2: Smooth Animation Curves

Instead of linear movement, use easing functions. Objects accelerate then decelerate like real physics.

LESSON 3: Velocity Vectors

Use (velocity_x, velocity_y) to move diagonally. Combine them for smooth movement in any direction.

LESSON 4: Deceleration and Friction

Add friction to slow down objects. Each frame: velocity *= 0.95 creates natural drag effect.

```
CODE EXAMPLE:  
velocity_x *= 0.95  
velocity_y *= 0.95
```

LESSON 5: Inertial Movement

When key is released, object continues moving then stops. Don't stop instantly - add momentum decay.

LESSON 6: Diagonal Running Animation

Play different animation frames based on direction. Running animation has 4-8 frames per direction.

LESSON 7: Multiple Animation States

Idle, Running, Jumping, Falling. Switch states based on input and physics state.

LESSON 8: Smooth Jumping Physics

Jump height = initial_velocity / (2 * gravity). Peak at 50% duration. Feels responsive like Mario.

LESSON 9: Air Control During Jump

Allow limited movement mid-air. Real games let you steer while jumping for skill-based movement.

LESSON 10: Dash and Momentum Mechanics

Quick burst movement with cooldown. Used in Duke Nukem style action games for advanced play.

[COMPLETE] PART 1 FINISHED! You've mastered advanced concepts for this area.

LESSON 11: Understanding Raycasting

DOOM used raycasting to create 3D illusion. Draw vertical lines at different heights.

LESSON 12: Simple 2.5D Technique

Draw walls taller based on distance. Closer = taller, farther = shorter. No real 3D calculations needed.

```
CODE EXAMPLE:  
wall_height = base_height / distance  
draw_wall(x, y, width, wall_height)
```

LESSON 13: Texture Mapping Basics

Apply 2D images to walls. Different textures for top, middle, bottom of walls.

LESSON 14: First-Person Perspective

Create a 3D-like view from player's eyes. Calculate what's visible from camera position.

LESSON 15: Wall Rendering in 2.5D

Draw vertical slices of the game world. Each slice represents a ray's distance to wall.

LESSON 16: Sprite Scaling in 3D

Enemies appear bigger when close, smaller when far. Scale sprite based on distance: $\text{sprite_size} = \text{original_size} / \text{distance}$

LESSON 17: Depth Sorting Objects

Draw far objects first, close objects last. This creates proper depth layering.

LESSON 18: Parallax Effect

Background moves slower than foreground. Creates depth illusion. Used in Command and Conquer.

LESSON 19: Billboards in 3D Space

2D sprites always face camera. Used for trees, enemies, objects in 3D games.

LESSON 20: Simple Overhead Map System

Display minimap showing game world. Helps player navigation in large levels.

[COMPLETE] PART 2 FINISHED! You've mastered advanced concepts for this area.

LESSON 21: Isometric Projection Math

3D coordinate to 2D screen: $\text{screen_x} = (x - y)$, $\text{screen_y} = (x + y) * 0.5$

CODE EXAMPLE:

```
screen_x = iso_x - iso_y  
screen_y = (iso_x + iso_y) * 0.5
```

LESSON 22: Isometric Tile Grid

Build world from grid of tiles. Each tile is diamond-shaped in isometric view.

LESSON 23: Isometric Sprite Orientation

Sprites face one of 8 directions in isometric view. Match sprite angle to world orientation.

LESSON 24: Drawing Order in Isometric

Sort by depth: draw tiles top-left to bottom-right. Prevents sprites from drawing incorrectly.

LESSON 25: Elevation in Isometric

Add height to tiles. Higher tiles drawn above lower tiles. Creates layered map.

LESSON 26: Isometric Building System

Rollercoaster Tycoon allows building structures. Check collision and elevation before placing.

LESSON 27: Isometric Camera Panning

Move camera smoothly across isometric map. Keep screen centered on player or selection.

LESSON 28: Isometric Pathfinding

Find path from point A to B on grid. Account for diagonal movement and elevation.

LESSON 29: Isometric UI Elements

Draw menus and buttons in isometric game. Keep UI readable despite perspective.

LESSON 30: Rotating Isometric View

Some games rotate the isometric grid. Recalculate all positions when view rotates.

[COMPLETE] PART 3 FINISHED! You've mastered advanced concepts for this area.

LESSON 31: Sprite Sheet Animation

Store multiple animation frames in single image. Use rect clipping to display frame by frame.

```
CODE EXAMPLE:  
frame = current_time // frame_duration  
rect = (frame * width, 0, width, height)
```

LESSON 32: Frame-Based Animation

Update animation frame every N ticks. Control animation speed independently of game speed.

LESSON 33: Particle Systems

Create many small sprites (particles) for effects. Explosions, fire, rain, smoke.

LESSON 34: Trail Effects

Draw line behind moving object. Reduces alpha each frame. Creates motion blur effect.

LESSON 35: Screen Shake Effect

Add small random offset to camera. Intensify on explosions or impacts.

LESSON 36: Color Shifting and Filters

Apply color overlay to sprites. Darken for shadows, tint for status effects.

LESSON 37: Lighting Systems

Darken sprites based on light distance. Create dynamic shadows and lit areas.

LESSON 38: Multiple Layers

Draw game in layers: background, terrain, objects, effects, UI. Control depth.

LESSON 39: Sprite Flipping and Mirroring

Flip sprites horizontally to face different directions. Saves memory vs storing separate images.

LESSON 40: Anti-Aliasing and Smoothing

Smooth edges on scaled sprites. Use `pygame.transform.smoothscale()` for better quality.

[COMPLETE] PART 4 FINISHED! You've mastered advanced concepts for this area.

LESSON 41: Simple AI Behavior States

Idle, Patrol, Chase, Attack. Switch states based on conditions.

CODE EXAMPLE:

```
if player_near: state = CHASE
elif at_patrol_end: state = IDLE
```

LESSON 42: State Machine Pattern

Formalize AI with explicit states and transitions. Clean code structure for complex behavior.

LESSON 43: Decision Trees for AI

If-then-else logic for NPC decisions. 'If player near? Chase. If tired? Rest.'

LESSON 44: Behavior Trees (Advanced)

Hierarchical AI system. Better than state machines for complex behavior.

LESSON 45: Line of Sight Detection

Can NPC see player? Cast ray from NPC to player. Check for obstacles.

LESSON 46: Pathfinding: A* Algorithm

Find optimal path from A to B. Account for obstacles. Core of strategy game AI.

LESSON 47: Tile-Based Pathfinding

Simplified A* on grid. Used in Rollercoaster Tycoon and Command and Conquer.

LESSON 48: Flocking and Group Behavior

Multiple NPCs coordinate movement. Create army units that move together naturally.

LESSON 49: Personality and Variation

Each NPC has slightly different stats. Speed, aggression, intelligence vary per unit.

LESSON 50: NPC Interaction Systems

NPCs talk to each other. Create dialogue trees and interaction prompts.

[COMPLETE] PART 5 FINISHED! You've mastered advanced concepts for this area.

LESSON 51: Follow Camera Basics

Camera center on player. Smooth follow vs snap-to-center.

CODE EXAMPLE:

```
camera_x = lerp(camera_x, player_x, 0.1)
camera_y = lerp(camera_y, player_y, 0.1)
```

LESSON 52: Bounded Camera

Keep camera within map bounds. Prevent showing outside world.

LESSON 53: Camera Smoothing

Lerp camera position for smooth motion. Don't snap instantly.

LESSON 54: Zoom and Unzoom

Allow player to zoom in/out. Smooth zoom transitions.

LESSON 55: Pan and Scroll

Drag camera with mouse. Used in strategy games like Command and Conquer.

LESSON 56: Focus Multiple Objects

Keep multiple objects in view. Calculate bounding box that includes all.

LESSON 57: Screen Edge Scrolling

Move camera when cursor near screen edge. Classic RTS game mechanic.

LESSON 58: Parallax Scrolling

Different layers scroll at different speeds. Background moves slower than foreground.

LESSON 59: Camera Shake Timing

Smooth screen shake that decays. More intense initially then settles.

LESSON 60: Cinematic Camera Paths

Define camera path points. Create cutscene camera movements.

[COMPLETE] PART 6 FINISHED! You've mastered advanced concepts for this area.

LESSON 61: Tile-Based Map System

Divide world into grid of tiles. Each tile has type: grass, wall, water.

```
CODE EXAMPLE:  
tile_type = tile_map[y // tile_size][x // tile_size]  
draw_tile(x, y, tile_type)
```

LESSON 62: Tile Collision Map

Separate data structure showing which tiles are walkable.

LESSON 63: Multi-Layer Terrain

Different tile layers: ground, decoration, overhead. Create depth.

LESSON 64: Procedural Map Generation

Create random maps algorithmically. Used in roguelike games.

LESSON 65: Perlin Noise for Terrain

Smooth random terrain generation. Mountains, valleys, plateaus.

LESSON 66: Dungeon Generation Algorithms

Create maze-like dungeons procedurally. Binary space partitioning.

LESSON 67: Tile Placement Rules

Ensure map is navigable. Check connectivity between tiles.

LESSON 68: Object Spawning Zones

Mark areas for enemies, items, NPCs. Spawn procedurally or from data.

LESSON 69: Level Editor Tools

Create custom levels easily. Visual tile placement interface.

LESSON 70: Saving and Loading Levels

Store level data to file. Load and recreate level from data.

[COMPLETE] PART 7 FINISHED! You've mastered advanced concepts for this area.

LESSON 71: Resource Management System

Track resources: gold, wood, energy. Gain resources over time.

```
CODE EXAMPLE:  
gold += gold_per_second / 60  
if gold >= building_cost: can_build = True
```

LESSON 72: Building Construction

Place buildings that cost resources. Buildings produce resources or unlock abilities.

LESSON 73: Unit Training and Spawning

Build units from buildings. Queue units for production like in RTS games.

LESSON 74: Fog of War System

Areas not visible are hidden. Reveals when units explore. Classic RTS mechanic.

LESSON 75: Unit Selection and Groups

Select single unit or group. Issue commands to selection.

LESSON 76: RTS-Style Command System

Right-click to move, attack, build. Left-click to select.

LESSON 77: Minimap with Visibility

Show terrain and units on minimap. Respect fog of war.

LESSON 78: Base Management

Track buildings, units, resources. Show status in UI. Like Rollercoaster Tycoon management.

LESSON 79: Research and Upgrades

Spend resources to unlock new abilities or improve units.

LESSON 80: Terrain Deformation

Terrain changes as you build. Dig paths, flatten land, create water features.

[COMPLETE] PART 8 FINISHED! You've mastered advanced concepts for this area.

LESSON 81: Gravity Implementation

Apply downward force each frame. $\text{velocity_y} += \text{gravity}$. Objects fall naturally.

```
CODE EXAMPLE:  
velocity_y += gravity  
position_y += velocity_y
```

LESSON 82: Terminal Velocity

Maximum falling speed. $\text{velocity_y} = \min(\text{velocity_y}, \text{max_fall_speed})$.

LESSON 83: Bounciness and Elasticity

Objects bounce on collision. $\text{velocity} *= -\text{bounce_factor}$. Different materials bounce differently.

LESSON 84: Rotation Physics

Objects spin based on angular velocity. Update rotation each frame.

LESSON 85: Friction Between Objects

Sliding friction opposes motion. Different surface types have different friction.

LESSON 86: Momentum Conservation

When objects collide, exchange momentum. Realistic collision physics.

LESSON 87: Force and Impulse Application

Apply instantaneous force on collision. $\text{Impulse} = \text{mass} * \text{velocity_change}$.

LESSON 88: Rigid Body Dynamics

Full physics: position, velocity, acceleration, rotation, angular velocity.

LESSON 89: Continuous Collision Detection

Detect collisions at intermediate frames. Prevent fast objects passing through walls.

LESSON 90: Soft Body Physics

Objects deform on impact. Cloth, jelly, flexible structures.

[COMPLETE] PART 9 FINISHED! You've mastered advanced concepts for this area.

LESSON 91: Game Architecture Patterns

Separate logic, rendering, input. Clean code organization.

```
CODE EXAMPLE:  
class Game:  
    def update(self): ...  
    def render(self): ...
```

LESSON 92: Entity Component System

Objects have components. Flexibility in behavior and data organization.

LESSON 93: Event System

Objects communicate via events. Decoupled communication between systems.

LESSON 94: Resource Manager

Load and cache resources (images, sounds, fonts). Avoid redundant loading.

LESSON 95: Game State Management

Handle game states: MENU, PLAYING, PAUSED, GAME_OVER. Clean transitions.

LESSON 96: Configuration Files

Store game settings in file. Easy tweaking without recompile.

LESSON 97: Debug Mode and Logging

Add debug overlay showing FPS, coordinates, collisions. Save logs for troubleshooting.

LESSON 98: Performance Optimization

Reduce draw calls, use object pooling, cull off-screen objects. Optimize hot paths.

LESSON 99: Cross-Platform Compatibility

Ensure game works on Windows, Mac, Linux. Test on each platform.

LESSON 100: Publishing Your Game

Build executable, create installer, distribute on itch.io or Steam. Share your work!

[COMPLETE] PART 10 FINISHED! You've mastered advanced concepts for this area.

YOU'RE NOW A GAME DEVELOPER!

You have completed all 200 lessons (Part 1 + Part 2)! You now have professional-level game development skills.

What You Can Build Now:

Action Games (like Duke Nukem, DOOM)

Strategy Games (like Command and Conquer, Rollercoaster Tycoon)

Complex Simulations

Isometric RPGs

Real-Time Management Games

Indie Games for Distribution

Recommended Next Steps:

1. Study professional game engines (Godot, Unity)
2. Join game dev communities and forums
3. Play classic games you learned from - study their design
4. Create your own original game concept
5. Contribute to open-source game projects
6. Consider a career in game development!
7. Keep learning and iterating on your skills

Happy Game Making! - Your journey as a game developer is just beginning.